AIE BOOTCAMP · FINAL CAPSTONE PROPOSAL · SOLO · ~2-WEEK AI-ASSISTED MVP

Keel

A sanctioned academic co-pilot that plans, predicts, advises — and safely acts.
The AI proposes the course; the keel keeps it upright.

Prepared by: _____ · **One backend, two surfaces, four capabilities:** Plan · Register · Advise · Predict · **Safety model:** intelligence proposes, the engine verifies, models predict, the student approves.

1 The Idea

Keel is a multi-tenant SaaS a university deploys for its students. It runs two React surfaces over one FastAPI backend: an **admin console** where the registrar grounds and configures the agent, and a **chat widget** embedded in the registration portal that students use. A student talks in plain language — *"15 credits, Fridays free, must take Data Structures, no 8 AMs"* — and Keel produces a valid, conflict-free plan, predicts whether that plan is *wise* (graduation risk and workload, not just legality), explains the trade-offs, and — only after the student approves — executes the enrollment.

It does four things and does them safely. It **plans** (next-semester, full path to graduation, what-if scenarios, and saved/named plans the student can compare and activate), it **registers** (finds open, conflict-free sections matching preferences, waitlists and notifies, and files real institutional requests such as petitions and graduation applications), it **advises** (grounded course information, prerequisite explanations, failure-recovery and major-switch reasoning), and it **predicts** (a trained graduation-risk model and a deterministic workload signal that score every candidate plan). Keel began as a registration copilot; it has matured into a student-success co-pilot — with planning and registration kept as first-class capabilities, not buried under new features.

2 The Problem It Solves

Registration and degree planning are universally painful — hidden time conflicts, prerequisite surprises, full sections, no advisor available at 11 PM, and no clear answer to *"am I on track to graduate?"* Students fall back on brittle, unsanctioned scraping bots; official advising chatbots only answer questions and route to a human. Keel is the sanctioned middle ground: an assistant that can **safely act** — with tenant isolation, human approval before any write, and a full audit trail — and that doesn't merely *tell* a student they are at risk, but *builds* them a legal, lower-risk plan, registers it on approval, and files the institutional paperwork that normally requires an office visit.

3 Core Design Principle — Propose · Verify · Predict · Explain

The defining rule: **the LLM never decides feasibility, and never predicts outcomes.** Three kinds of work live in three layers. Hard constraints (prerequisites, time conflicts, capacity, credit caps, offering term, holds) are owned by a **deterministic engine** and are non-negotiable. Outcome estimates (graduation risk, workload) are owned by **prediction models**. Everything fuzzy, conflicting, and underspecified — what the student actually wants, which plan to propose, how to rank and explain it — is where the **LLM** genuinely decides.

The LLM decides (soft · fuzzy · generative)	**The engine guarantees** (hard · exact · verified)
Intent classification (when ambiguous) · constraint & preference extraction from natural language · proposing candidate plans, elective sets, orderings · ranking and explaining trade-offs · repairing a plan from structured violations · drafting petitions and request justifications · advisory recommendations.	Prerequisite ordering (DAG topological check) · time-conflict detection · section capacity · credit caps · offering-term cadence · corequisites · registration holds · eligibility · workload aggregation (difficulty index) · idempotent, transactional enrollment write.

The models predict (probabilistic · learned)	**How the three layers meet**
Graduation-risk classifier (trained, compared ML vs DL vs LLM, with class-imbalance handling) scores a *valid* plan as on-track / at-risk and feeds a mitigation explanation. **GPA estimate** is an explicit LLM baseline — the weak, uncalibrated option, framed as such. **Workload** is deterministic, not a model.	A **generate–verify–repair loop**: the LLM proposes, the engine validates and returns structured violations, the LLM repairs and re-proposes; valid candidates are then *scored* by the models, and the LLM ranks and explains using both feasibility and risk. A greedy deterministic planner sits behind it as a fallback.

Intelligence proposes. Deterministic systems verify. Models predict. Execution requires approval.

4 What Keel Does — Features

Every feature reuses a small set of engines (the deterministic core, the advising RAG, the LLM, the model-server) and a single **action pattern** — propose → engine-validate → human-approve → idempotent transactional write → audit. The modules below are *intents and actions over those engines*, not independent pipelines. Each card states why it exists, its constraints, how AI is used, and what stays deterministic.

MODULE A — PLANNING

A1 Next-Semester Planning — MVP

PURPOSE Answer “what should I take next term?” and feed registration. Core of the demo.

CONSTRAINTS Plan must satisfy prerequisites, credit caps, corequisites, no repeats of passed courses, degree progress.

AI LLM extracts hard constraints + soft preferences, proposes 2–3 candidate plans (balanced / graduation-focused / lighter), ranks and explains.

DETERMINISTIC Audit engine builds the eligible course pool; verifier validates every candidate and returns structured violations.

A2 Graduation Planning — MVP

PURPOSE Map the whole path from current term to graduation — the “am I on track, and how?” answer.

CONSTRAINTS Full multi-term path must respect the prerequisite DAG, offering cadence, and per-term credit limits end-to-end.

AI LLM optimizes the skeleton toward a goal (fastest / balanced / easier terms) and explains the sequence.

DETERMINISTIC Engine computes remaining requirements and credits, generates the path skeleton, validates the whole path.

A3 What-If Simulation — MVP

PURPOSE “What if I switch to Math?” / “Can I graduate a semester early?” — re-audit under modified assumptions.

CONSTRAINTS Returns graduation date + required credits-per-term; never presents an infeasible alternative as feasible.

AI LLM explains consequences and trade-offs of the change.

DETERMINISTIC Engine recomputes audit against the target program and derives the new timeline.

A4 Saved Plans — Save / Load / Activate — MVP

PURPOSE Persist named plans (“Fast Graduation”, “Easy Semester”, “Career-aligned”); load, compare, and activate one. Foundational — swap, replanning, and invalidation all build on it.

CONSTRAINTS A first-class, **versioned Plan entity** with exactly one active plan; every saved plan was verifier-valid at save time and is re-validated on load if the catalog changed.

AI None for storage; the LLM names and explains plans on request.

DETERMINISTIC Plan entity, versioning, activation, and the comparison view.

A5 Course Swap — ADVANCED

PURPOSE “Replace Database with Networks” → validate → update the active plan.

CONSTRAINTS The swap must keep the plan verifier-valid (prerequisites, conflicts, credits); reject and explain if it cannot.

AI LLM interprets the request and explains the consequence of the swap.

DETERMINISTIC Engine re-validates the edited plan; idempotent transactional update on the Plan entity.

A6 Automatic Replanning — ADVANCED

PURPOSE A saved plan becomes invalid (course removed, prerequisite changed, section cancelled) → re-audit, re-plan, notify the student. A strong engineering feature.

CONSTRAINTS Scope which saved plans are actually affected; recompute must converge or flag for a human; avoid notification storms.

AI LLM explains what changed and what the recompute did.

DETERMINISTIC Background worker re-runs audit + planner on affected plans; transactional write + **outbox** notification.

MODULE B — REGISTRATION

B1 Registration Assistant — MVP

PURPOSE Turn an approved plan (or manually named courses) into a real, conflict-free enrollment.

CONSTRAINTS A phased, bounded workflow — not an open agent loop. Engine owns eligibility; the write requires explicit approval and is transactional + idempotent.

AI LLM interprets schedule preferences and explains the section options. Limited, deliberately.

DETERMINISTIC Section search returns open, conflict-free combinations; idempotent enrollment write executes on approval.

B2 Waitlist (Join / Leave) & Seat Tracking — MVP

PURPOSE	Full section? Join or leave a waitlist as first-class actions; “notify me when a seat opens.”
CONSTRAINTS	Email-on-seat-open retries with backoff and never breaks the user-facing response.
AI	None — pure workflow.
DETERMINISTIC	Background RQ worker polls capacity; transactional waitlist writes; notification via the outbox.

MODULE C — ADVISING

C1 Course Advisor (RAG) — MVP

PURPOSE	Answer what a course covers, what it unlocks, and what it requires — grounded, never invented.
CONSTRAINTS	Prerequisite chains are grounded in the DAG, so the model cannot invent a prerequisite.
AI	Hybrid retrieval + rerank over the tenant’s catalog and policy documents; LLM answers from context with sources.
DETERMINISTIC	Retrieval is tenant-filtered; prerequisite facts come from the DAG, not the prose.

C2 Degree Audit Chat — MVP

PURPOSE	“What do I still need to graduate?” in plain language.
CONSTRAINTS	Numbers come from the engine; the LLM may not restate them incorrectly.
AI	LLM summarizes the audit result conversationally.
DETERMINISTIC	Engine computes missing requirements, remaining credits, and required courses.

C3 Academic Advisor Chat — Failure Recovery — ADVANCED

PURPOSE	“I failed Data Structures. Am I doomed?” — turn a setback into a concrete recovery plan.
CONSTRAINTS	Recovery plan must itself pass the verifier; graduation-impact figures come from the engine.
AI	LLM composes the recovery narrative and re-plan rationale. Genuine reasoning + explanation.
DETERMINISTIC	Engine computes downstream delayed courses and the graduation-date impact.

C4 Major-Switch Advisor — ADVANCED

PURPOSE	“Should I switch majors?” — adds a recommendation layer on top of A3’s what-if. (The *action* to file the switch lives in F2.)
CONSTRAINTS	Consequences (lost credits, new timeline) are engine-computed; recommendation is advisory, not a guarantee.
AI	LLM analyzes performance patterns and strengths and frames a recommendation.
DETERMINISTIC	Engine computes graduation consequences against each candidate program.

MODULE D — PREDICTION (the layer instructors asked to see)

D1 Graduation-Risk Predictor — ADVANCED

PURPOSE	“Am I on track?” — the headline ML feature. Scores a valid plan on-track / at-risk and drives a mitigation plan.
CONSTRAINTS	A **trained** model (one of the two trained models), compared three ways — classical ML vs a small DL model (ONNX) vs an LLM baseline — macro-F1 gates CI, with **class-imbalance handling** and per-class recall reported (at-risk is the minority class you care about). Served lean (no torch in containers).
AI	ML model produces the risk score + reasons; the LLM generates the mitigation plan from those reasons.
DETERMINISTIC	Inputs (GPA trajectory, failed courses, progress rate, current load) are computed by the engine.

D2 Workload Signal — ADVANCED

PURPOSE	“How heavy is this semester?” — the second half of “is this plan *wise*?”
CONSTRAINTS	**Deterministic by design** — a difficulty-weighted aggregation (per-course difficulty × credits → light / medium / heavy). Not a third trained model, to keep training scope at two.
AI	None for the score; the LLM only narrates it inside plan explanations.
DETERMINISTIC	Engine aggregates the difficulty index. (May become a trained model later — see Data Sources, TBA.)

D3 GPA Estimate — ADVANCED

PURPOSE	“What GPA might I get with this plan?” — a light, advisory estimate.
CONSTRAINTS	Implemented as an **LLM baseline** and labelled as the weak, uncalibrated option — a live example of the “when *not* to use an LLM” lesson. Not a headline; never a guarantee.
AI	LLM estimate from transcript + course difficulty + load. Swappable for a model later.
DETERMINISTIC	Feature inputs are engine-computed; the estimate is explicitly bounded and caveated.

MODULE E — GUIDANCE

E1 Personalized Elective Recommender

ADVANCED

PURPOSE Rank electives by fit (strengths, GPA goals, difficulty preference, career direction).

CONSTRAINTS Only recommends electives that **exist in the catalog and are eligible** — grounded in the DAG + audit, so it cannot suggest a course the student can't take.

AI LLM ranks and justifies using the transcript strengths/weaknesses analysis.

DETERMINISTIC Eligible elective set comes from the engine; strengths/weaknesses from transcript analysis.

E2 Career Path Recommender

ADVANCED

PURPOSE "I want to become an AI Engineer" → map interests + strengths to a direction and the catalog electives/skills that align. The resulting roadmap can be saved as a career-aligned plan via A4 STRETCH.

CONSTRAINTS The one feature with **no hard verifier and no ground truth** — framed explicitly as a *suggestion*, not a prediction. Recommendations are **grounded in the real catalog** so it cannot invent courses; it feeds E1 and the planning goal.

AI LLM maps career interest → relevant skills → recommended catalog electives → projects.

DETERMINISTIC Course/skill grounding is catalog-bound via RAG + DAG.

MODULE F — INSTITUTIONAL REQUESTS (routed to the registrar in the admin console — no extra role)

F1 Graduation Application

ADVANCED

PURPOSE "Am I ready to graduate?" → audit confirms eligibility → apply for graduation. The natural terminal action of the planning arc.

CONSTRAINTS The engine confirms all requirements are met before the action is offered; the write is idempotent and audit-logged, routed to the registrar.

AI LLM explains readiness; it does not decide eligibility.

DETERMINISTIC Audit eligibility check; application write to the institutional-request queue.

F2 Major-Change Request

ADVANCED

PURPOSE "Switch to Data Science" → impact analysis (C4) → submit a major-change request to the registrar.

CONSTRAINTS Consequences are engine-computed; the request is a routed write, not an auto-approved change.

AI LLM frames the impact summary attached to the request.

DETERMINISTIC Engine computes lost credits / new timeline; request write + audit.

F3 Petition / Prerequisite Override

ADVANCED

PURPOSE Student wants a course they don't satisfy prerequisites for → engine explains the block → LLM drafts a justified petition → submit an exception request.

CONSTRAINTS The engine still **refuses to auto-enroll**; the petition is a request *to a human*, not a bypass — the seatbelt gains a sanctioned override path, it is not removed.

AI LLM drafts the petition from the student's stated justification + transcript context.

DETERMINISTIC Eligibility-block detection; petition write routed to the registrar; audit.

F4 Advisor Escalation — Email + Handoff Summary

ADVANCED

PURPOSE "I want to talk to an advisor" / an out-of-scope turn → decide escalation → generate a handoff summary → email it to the advisor.

CONSTRAINTS **No advisor role, login, or dashboard.** Escalation is an email carrying the summary; an optional appointment request is a recorded row, not a calendar integration.

AI LLM decides escalation and writes the handoff summary — conversation recap, transcript summary, failed constraints, recommended actions.

DETERMINISTIC Escalation routing; email via the outbox; audit.

MODULE G — PROACTIVE & MULTILINGUAL

G1 Personalized Alerts

ADVANCED

PURPOSE Proactive notifications: a seat opened, you can now take X (eligibility unlocked), graduation risk crossed a threshold, the registration window is opening.

CONSTRAINTS A **fixed small set of triggers (~4)**, not a general rules engine; deduplicated; delivered in-app and by email.

AI LLM phrases the alert; the triggers themselves are deterministic.

DETERMINISTIC Background worker evaluates trigger rules; notification table; outbox delivery.

G2 Multilingual Support

STRETCH

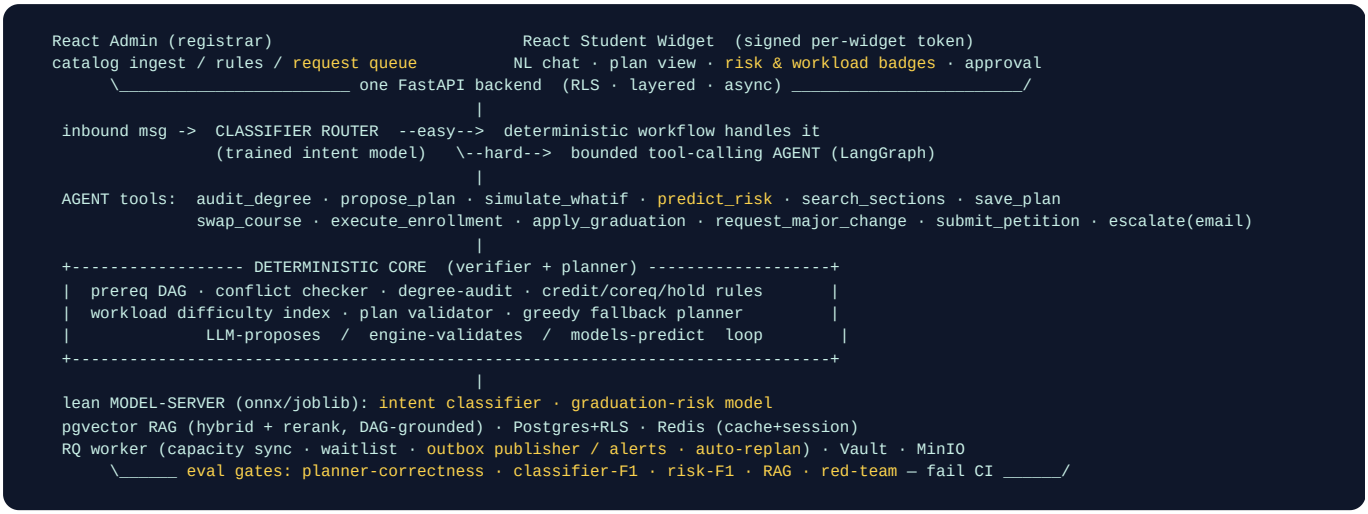
PURPOSE	Arabic / French / English — locally relevant for the deploying institution.
CONSTRAINTS	Low architectural complexity — prompt + UI; engine and predictions are language-agnostic.
AI	LLM responds in the student's language; retrieval stays grounded.
DETERMINISTIC	No change to the engine, verifier, or writes.

5 Roles & Access

Role	Powers
Admin (registrar)	Grounds the agent — uploads catalog & policy docs; sets registration rules (credit caps, holds, windows); configures persona, enabled tools, guardrails; manages the widget embed snippet and allowed origins; reviews audit log and per-tenant cost; and **works the institutional-request queue** (graduation applications, major-change requests, petitions).
Student	Uses the chat widget — asks in plain language, reviews the proposed conflict-free plan, its predicted risk/workload, and trade-offs, saves and compares plans, and approves before any enrollment or request executes. Scoped to their own identity and transcript.
Platform operator	Provisions, suspends, and erases university tenants. Never reads a tenant's conversations or data — a controlled doorway, not god mode. Every action is audit-logged.

Three named roles, fixed powers — deliberately no configurable RBAC matrix and no separate advisor role. Advisor escalation is an **email with a handoff summary**, not a login; institutional requests are worked by the registrar in the existing admin console.

6 Architecture Overview



Key subsystems

- **Plan entity** — a first-class, versioned, named plan with one active version. Underpins save/load/activate (A4), course swap (A5), and automatic replanning (A6).
- **Institutional-request queue** — one inbox the registrar works for graduation applications, major-change requests, and petitions (F1–F3). One subsystem, not four features, and no new role.
- **Action pattern + outbox** — every side-effecting write (enrollment, waitlist, requests, alerts) follows propose → validate → approve → **transactional write + outbox event** → audit. The outbox avoids dual-write inconsistency between the write and its notification/email.

7 The Planning Loop (refined)

The linear pipeline becomes a **generate–verify–predict–explain loop**, with prediction inserted only after feasibility is proven (no point scoring an illegal plan):

Student request → LLM extracts intent + constraints + preferences → engine builds the eligible course pool → **LLM proposes** candidate plans → **engine validates** & returns structured violations → *(repair loop: LLM repairs & re-proposes until valid)* → **models predict** graduation-risk & workload on the valid candidates → **LLM ranks & explains** using feasibility + risk → student overrides → engine re-validates & re-scores → **student approves** → **idempotent, transactional enrollment write** (+ outbox notification) → the approved plan is saved and monitored for invalidation. Greedy deterministic planner as fallback if the loop fails to converge.

8 Core AI Components

- **Intent classifier** (register / plan / advise / predict) — a real **trained** model compared three ways (classical ML, a small DL model exported to ONNX, an LLM zero-shot baseline); macro-F1 gates CI. Kept trained on purpose: the router exists to keep the LLM off cheap, enumerable decisions.

- **Graduation-risk model** — the second **trained** model; same three-way comparison; class-imbalance handling; risk-F1 (and at-risk recall) gates CI; feeds the LLM mitigation plan.
- **Constraint & preference extraction** — structured Pydantic outputs turn free text into typed hard constraints and soft preferences.
- **Plan proposal & repair** — the LLM proposes candidates inside the generate-and-verify loop and repairs them from the engine's structured violations.
- **Advising RAG** — hybrid retrieval + rerank over catalog & policy, with faithful, **DAG-grounded** prerequisite explanations.
- **Drafting & explanation** — petition drafts, handoff summaries, trade-off explanations, strengths/weaknesses, mitigation, and recommendations.
- **Guardrails** — prompt-injection / cross-tenant refusal and PII redaction before anything leaves the service boundary.

9 Core Software-Engineering Components

- **Multi-tenant isolation** — Postgres Row-Level Security + repository-layer scoping + tenant-filtered pgvector. *Isolation is the grade.*
- **Layered backend** (api / services / repositories / domain / infra), async throughout, dependency injection, lifespan singletons, typed boundaries.
- **First-class Plan entity** (versioned, named, active) and the **institutional-request queue** worked by the registrar — no extra role.
- **Transactional writes + outbox pattern** for the write-and-notify path (enrollment, waitlist, requests, alerts) to avoid dual-write inconsistency; idempotency keys; an audit-log row on every write.
- **Background jobs** — an RQ / Redis worker for capacity sync, waitlist processing, email-on-seat-open, personalized alerts, and automatic replanning (retry + backoff, structured logging).
- **Caching** with deliberate invalidation on catalog change; per-tenant rate limiting.
- **Widget auth** — signed, short-lived per-widget token + server-side origin check (CORS / CSP as defense-in-depth, never the boundary).
- **Secrets in Vault**, blob in MinIO, OpenTelemetry / LangSmith tracing, Alembic migrations, `docker-compose up` from a clean clone, GitHub Actions CI with eval gates.

10 MVP · Advanced · Stretch

Tier	Scope
MVP the demo spine	Intent classifier + router · constraint extraction · next-semester / graduation / what-if planning (propose→verify→repair loop) · **first-class Plan entity** with save/load/activate · registration (section search + idempotent transactional execute + approval) · waitlist join/leave + seat-tracking + email worker · Course Advisor RAG (hybrid + rerank) · Degree Audit chat · multi-tenant RLS · widget signed-token auth · Vault · layered backend · outbox writes · CI eval gates (planner-correctness + classifier-F1 + RAG + red-team).
ADVANCED	Graduation-risk model + LLM mitigation · failure-recovery chat · major-switch advisor · personalized elective recommender · GPA estimate (LLM) · workload signal (deterministic) · course swap · automatic replanning · institutional requests (graduation application, major-change request, petition/override) · advisor escalation email + handoff summary · personalized alerts (capped triggers) · career-path recommender (catalog-grounded).
STRETCH	Multilingual support (Arabic / French / English) · career roadmap saved as a plan · NeMo Guardrails sidecar (vs. lightweight in-process rails for MVP).

11 Evaluation & Quality Gates (fail CI on regression)

- **Planner correctness** — the headline gate. No generated plan ever violates a prerequisite, time conflict, capacity, credit cap, or hold; the verifier proves it on a golden set.
- **Intent classifier** — macro-F1 on the held-out test set, with the ML / DL / LLM three-way comparison committed alongside.
- **Graduation-risk model** — macro-F1 *and per-class recall on the at-risk minority* on a held-out split; three-way comparison committed; the model card discloses the data source (see §12).
- **Agent tool-selection** — given a message, did it pick the right tool (or correctly pick none)?
- **RAG** — retrieval (hit@k, MRR) and generation (faithfulness, relevancy) on a hand-curated golden set.
- **Red-team / write-action safety** — cross-tenant and prompt-injection probes must all be refused; a fake key never leaks unredacted; and **no injected or unapproved request ever produces a write** (enrollment, petition, major-change, graduation application). Testing the action pattern once covers every action.

12 Data Sources & Honesty TBA

- **Catalog & policy documents** — tenant-provided; a mock catalog with a real prerequisite DAG is seeded for the demo.
- **Intent dataset** — a small labeled text-classification set (public or hand-curated), held out with no leakage.
- **Graduation-risk data** — **TBA.** Preferred path: a suitable public student-performance / at-risk dataset, if one fits the feature schema. Fallback: **synthetic data, generated honestly**, with explicit class-imbalance handling (the at-risk class is the minority).

> **On synthetic data — the framing that survives a defense.** Real transcript data is privacy-protected (FERPA), which is the legitimate reason to synthesize. The data is generated with a **deliberate, documented signal plus injected noise** — never "purely random" (random data is unlearnable, which would make the model useless). The model card states plainly that the model learns *the generator's notion of risk, not validated real-world risk*; a small slice is hand-labeled and reported; macro-F1 and at-risk recall are gated on a held-out split. This is the Week-3 "weak supervision, documented honestly" lesson applied directly.

13 Tech Stack (every choice justified; nothing decorative)

Technology	Why it earns its place
React (admin + widget)	Two distinct surfaces; the widget is the production-shaped, embeddable student UI.
FastAPI + Pydantic	Async API, typed boundaries, structured LLM/tool I/O.
LangGraph bounded agent	Tool-calling agent for the *hard* turns only; bounded loop = cost + safety control.
Deterministic engine	The core of the project — DAG, conflict checker, audit, validator, greedy fallback. The differentiator lives here.
Postgres + RLS + Alembic	One DB for everything; RLS enforces tenant isolation at the database. Hosts the Plan entity, request queue, outbox, and audit log.
pgvector (hybrid + rerank)	Advising RAG in the same database, tenant-filtered by construction.
Model-server (ONNX / joblib)	Intent + graduation-risk served lean — **no torch in any container**; train offline, serve light.

Technology	Why it earns its place
Redis + RQ worker	Session memory + cache; background capacity sync, waitlist, alerts, outbox publishing, auto-replan.
Signed per-widget token	Real widget auth; CORS/CSP are defense-in-depth, never the boundary.
Vault · MinIO	Secrets at startup; blob for model artifacts, eval reports, snapshots.
Email (Resend / SMTP)	Seat-open notifications, alerts, and advisor-escalation emails — all via the outbox.
Docker Compose · GitHub Actions	One-command stack from a clean clone; CI with eval gates that fail on regression.
OpenTelemetry / LangSmith	Traces over every LLM/tool/retrieval call; joinable with redacted logs.
NeMo Guardrails (sidecar) STRETCH	Topical + injection rails as a sidecar; a lightweight in-process layer covers the MVP.

Deliberately not adopted (and why) — MLflow registry / drift / retraining pipeline: overkill for two small static models on synthetic data; model cards + SHA-256 + refuse-to-boot + CI eval gates already provide controlled promotion at the right scale. **MCP / A2A multi-agent:** would contradict the deliberate single-bounded-agent design — one agent behind a trained router is the correct, cheaper choice. **GraphRAG:** the prerequisite DAG is already used *deterministically* by the engine, which is stricter and more accurate than approximating it with graph retrieval. **Benchmarks/leaderboards, enterprise connectors, fine-tuned large transformer:** no real fit — each would cost build time without serving the project.

14 Deliverables

A public GitHub repository that comes up with a single `docker-compose up` from a clean clone: React admin + student widget, FastAPI backend, the deterministic engine, the lean model-server, a mock registration database and portal, seeded tenants, CI with eval gates, and the documentation set — `SPEC.md` (component specs written before the code), `DESIGN.md`, `DECISIONS.md`, `PLANNER.md` (the engine + verifier + outbox contract), `DATA.md` (the synthetic-data honesty note), `EVALS.md`, `RUNBOOK.md`, `SECURITY.md` — plus a short demo video of one end-to-end run from the widget through to a safely executed enrollment and a filed institutional request.

15 Scope Notes & Risks

- **The deterministic engine is the real time sink — and the core value.** The DAG, conflict checker, audit, and the generate-verify-repair loop are the hardest, least AI-assistant-friendly parts and they exist before any expansion. Build the engine correct first; a leaky verifier undermines everything above it.
- **Two trained models, no more.** Intent + graduation-risk. GPA is a light LLM baseline; workload is deterministic. Resisting a third training job is a deliberate choice, not a gap.
- **Action sprawl lives in the test surface, not the code.** Every approval-gated write is something an injection could try to trigger. The single action pattern (validate → approve → transactional write + outbox + audit) is tested once and reused, so adding actions stays cheap and safe.
- **Career path is the soft spot.** No verifier, no ground truth — framed as a grounded *suggestion*, with a prepared “I don’t claim correctness here” answer.
- **Richness without sprawl.** The features are intents and actions over a few engines plus two new subsystems (Plan entity, request queue), not independent pipelines. Depth on the planning loop, the predict layer, and the safe-write path beats breadth across half-built features.